

Computation Structures

Compilation Worksheet

compile_expr(expr) $\Rightarrow$ Rx

- Constants: $1234 \Rightarrow Rx$
 - `CMOVE(1234, Rx)`
 - `LD(c1, Rx)`
 - ...
 - `c1: LONG(123456)`
- Variables: $a \Rightarrow Rx$
 - `LD(a, Rx)`
 - ...
 - `a: LONG(0)`
- Variables: $b[expr] \Rightarrow Rx$
 - `compile_expr(expr)  $\Rightarrow Rx$`
 - `MULC(Rx, bsize, Rx)`
 - `LD(Rx, b, Rx)`
 - ...
 - `// reserve array space`
 - `b: . = . + bsize*blen`
- Operations: $expr_1 + expr_2 \Rightarrow Rx$
 - `compile_expr(expr1)  $\Rightarrow Rx$`
 - `compile_expr(expr2)  $\Rightarrow Ry$`
 - `ADD(Rx, Ry, Rx)`
- Procedure call: $f(e_1, e_2, \dots) \Rightarrow Rx$
next lecture!
- Assignment: $a=expr \Rightarrow Rx$
 - `compile_expr(expr)  $\Rightarrow Rx$`
 - `ST(Rx, a)`

compile_statement(...)

- Unconditional: $expr;$
 - `compile_expr(expr)`
- Compound: $\{ s1; s2; \dots \}$
 - `compile_statement(s1)`
 - `compile_statement(s2)`
 - ...
- Conditional: $\text{if } (expr) s1;$
 - `compile_expr(expr)  $\Rightarrow Rx$`
 - `BF(Rx, Lendif)`
 - `compile_statement(s1)`
 - `Lendif:`
- Conditional: $\text{if } (expr) s1; \text{ else } s2;$
 - `compile_expr(expr)  $\Rightarrow Rx$`
 - `BF(Rx, Lelse)`
 - `compile_statement(s1)`
 - `BR(Lendif)`
 - `Lelse:`
 - `compile_statement(s1)`
 - `Lendif:`
- Iteration: $\text{while } (expr) s1;$
 - `BR(Ltest)`
 - `Lwhile:`
 - `compile_statement(s1)`
 - `Ltest:`
 - `compile_expr(expr)  $\Rightarrow Rx$`
 - `BT(Rx, Lwhile)`
- Iteration: $\text{for } (init; test; incr) s1;$
 - `init;`
 - `while (test) { s1; incr; }`

Problem 1.

Please hand-compile the following snippets of C code into equivalent Beta assembly language statements. Assume that memory locations have been allocated for the all C variables with labels that corresponds to the variable names. So to load the value of the C variable *a* into register R3, the appropriate assembly language statement would be `LD(R31, a, R3)`. And to store the value in R17 to the C variable *b*, the appropriate assembly language statement would be `ST(R17, b, R31)`. Similarly, assume that memory locations have been allocated for each C array, with a label defined whose value is the address of the 0th element of the array.

(A) `a = 42;`

```
CMOVE(42, R1)
ST(R1, a, R31)
```

(B) `c = 5*x - 13;`

```
CMOVE(5, R0)
LD(X, R1)
MUL(R0, R1, R0)
CMOVE(13, R1)
SUB(R0, R1, R2)
ST(R2, c, R31)
```

(C) `y = (x - 3)*(y + 123456);`

```
LD(X, R1)
SUBC(R1, 3, R1)
LD(y, R2)
LD(123456, R0)
ADD(R0, R2, R2)
MUL(R1, R2, R1)
ST(R1, y)
```

(D) `if (a == 3) b = b + 1;`

```
LD(a, R0)
CMPEQC(R0, 3, R1)
BF(R1, L1)
LD(b, R0)
ADDC(R0, 1, R0)
ST(R0, b)
L1:
```

(E) `a[i] = a[i-1];`

```
LD(i, R0)
MULC(R0, 4, R0) //R0 = 4i
LD(R0, a - 4, R1) //R1 = 4i + a - 4 (địa chỉ của a[i-1])
ST(R1, a, R0) // R1 lưu vào địa chỉ 4i + a (a[i])
```

(F) `x = y[3] + y[12];`

```
LD(y + 3*4, R0) | R0 = y[3]
LD(y + 12*4, R1) | R1 = y[12]
ADD(R0, R1, R0)
ST(R0, x)
```

(G) `if (b == 0 || b < min) {`

```
    min = b;
} else {
    too_big += 1;
}
```

```
Optimize:
LD(b, R0)
BEQ(R0, L1)
LD(min, R1)
CMPLTC(R0, R1, R2)
BF(R2, L2)
L1:
    ST(R0, min)
L2:
    LD(too_big, R5)
    ADDC(R5, 1, R5)
    ST(R5, too_big)
```

Unoptimize:

```
LD(b, R0)
LD(min, R2)
LD(too_big, R5)
CMPEQC(R0, 0, R1)
CMPLTC(R0, R2, R3)
OR(R1, R3, R4)
BF(R4, L1)
ST(R0, min)
BR(L2)
L1:
    LD(too_big, R5)
    ADDC(R5, 1, R5)
    ST(R5, too_big)
L2:
```

(H) `sum = 0;`

```
    i = 0;
    while (i < 10) {
        sum = sum + i
        i = i + 1;
    }
```

```
LD(sum, R0)
LD(i, R1)
L1:
    CMPLTC(R1, 10, R2)
    BF(R2, L3)
L2:
    ADD(R0, R1, R0)
    ADDC(R1, 1)
    ST(R0, sum)
    ST(R1, i)
    BR(L1)
L3:
```

Problem 2.

In block-structured languages such as C or Java, the scope of a variable declared locally within a block extends only over that block, i.e., the value of the local variable cannot be accessed outside the block. Conceptually, storage is allocated for the variable when the block is entered and deallocated when the block is exited. In many cases, this means the compiler is free to use a register to hold the value of the local variable instead of a memory location.

Consider the following C fragment:

```
int sum = 0;
{ int i;
  for (i = 0; i < 10; i = i+1) sum += i;
}
```

- A. Hand-compile this loop into assembly language, using registers to hold the values of the local variables "i" and "sum".

```
CMOVE(0, R0)
CMOVE(0, R1)
ST(R0, sum)
L1:
CMPLTC(R1, 10, R2)
BF(R2, L2)
ADD(R0, R1, R0)
ADDC(R1, 1, R1)
BR(L1)
L2:
ST(R0, SUM)
```

- B. Define a *memory access* as any access to memory, i.e., instruction fetch, data read (LD), or data write (ST). Compare the number of total number of memory accesses generated by executing the optimized loop with the total number of memory access for the unoptimized loop (part G of the preceding problem).

Unoptimize - worst case: 10 instruction fetch, 5 data memory acces => 15 acces

Optimize - worst case: 8 instruction fetch, 4 memory acces => 12 acces

- C. Some optimizing compilers "unroll" small loops to amortize the overhead of each loop iteration over more instructions in the body of the loop. For example, one unrolling of the loop above would be equivalent to rewriting the program as

```
int sum = 0;
{ int i;
  for (i = 0; i < 10; i = i+2) {
    sum += i; sum += i+1;
  }
}
```

```
CMOVE(0, R0)
ST(R0, sum)
CMOVE(0, R1)
L1:
ADD(R0, R1, R0)
ADDC(R1, 1, R1)
ADD(R0, R1, R0)
ADDC(R1, 1, R1)
CMPLTC(R1, 10, R2)
BT(R2, L1)
ST(R0, sum)
```

Hand-compile this loop into Beta assembly language and compare the total number of memory accesses generated when it executes to the total number of memory accesses from part (1).

New: each iteration

6 instruction fetches

5 iteration => 34 instruction fetchs + 2 data memory acces => 36

Old: ech iteration

5 instruction fetches

10 iteration => 50 instruction fetchs + 8 data memory acces => 52

Problem 3.

Which of the following Beta instruction sequences might have resulted from compiling the following C statement? For each sequence describe the value that does end up as the value of y.

```
int x[20], y;  
y = x[1] + 4;
```

A. LD (R31, x + 1, R0) LD(x + 1*4, R2)
 ADDC (R0, 4, R0) ADDC(R2, 4, R2)
 ST (R0, y, R31) ST(R2, y)

B. CMOVE (4, R0)
 ADDC (R0, x + 4, R0)
 ST (R0, y, R31)

C. LD (R31, x + 4, R0)
 ST (R0, y + 4, R31)

D. CMOVE (4, R0)
 LD (R0, x, R1)
 ST (R1, y, R0)

E. LD (R31, x + 4, R0)
 ADDC (R0, 4, R0)
 ST (R0, y, R31)

F. ADDC (R31, x + 1, R0)
 ADDC (R0, 4, R0)
 ST (R0, y, R31)

MIT OpenCourseWare
<https://ocw.mit.edu/>

6.004 Computation Structures
Spring 2017

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>.